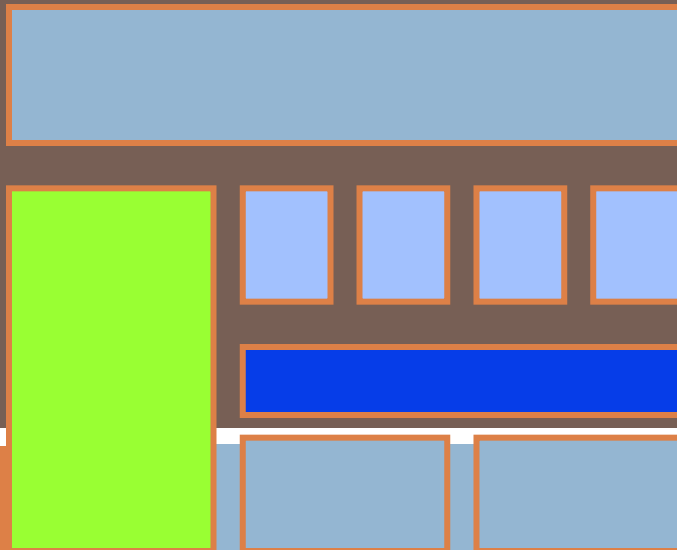


FUNDAMENTALS OF PROGRAMMING

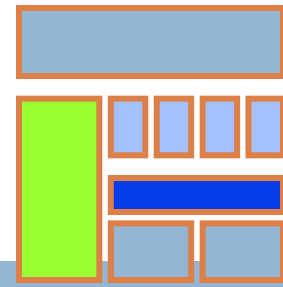
Lecture 13: STRUCTURE

Instructor: Dr Aqib Perwaiz

STRUCTURES

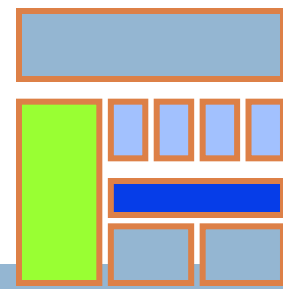


Structures



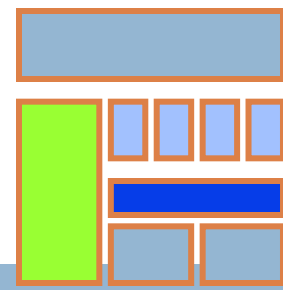
- A **Structure** is a collection of related data items, possibly of different types.
- A structure type in C++ is called **struct**.
- A **struct** is **heterogeneous** in that it can be composed of data of different types.
- In contrast, **array is homogeneous** since it can contain only data of the same type.

Structures



- Structures hold data that belong **together**.
- Examples:
 - ▣ Student record: student id, name, major, gender, start year, ...
 - ▣ Bank account: account number, name, currency, balance, ...
 - ▣ Address book: name, address, telephone number, ...
- In database applications, structures are called records.

Structures

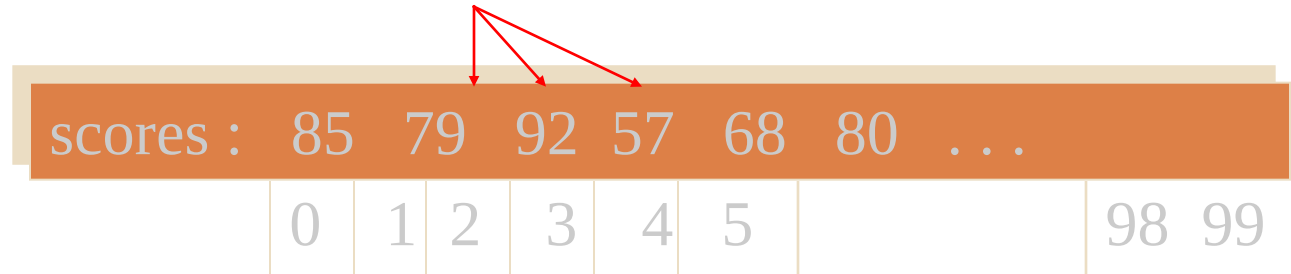


- Individual components of a struct type are called **members (or fields)**.
- Members can be of **different types** (simple, array or struct).
- A struct is named as a whole while individual members are named using field identifiers.
- Complex data structures can be formed by defining **arrays of structs**.

Structures

6

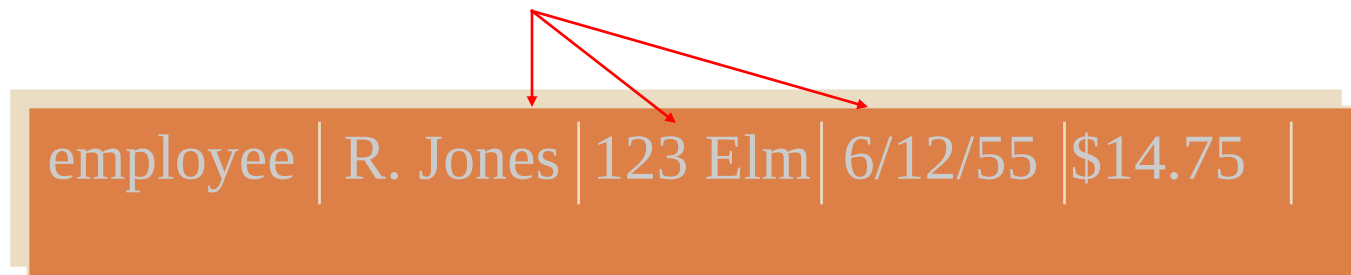
- Recall that elements of arrays must all be of the same type



A diagram illustrating an array of integers. A horizontal bar is divided into cells. The first cell contains the text 'scores :'. The subsequent cells contain the values 85, 79, 92, 57, 68, and 80, followed by an ellipsis '...'. Below the bar, indices 0, 1, 2, 3, 4, and 5 are aligned under the corresponding values. To the right of the ellipsis, the indices 98 and 99 are shown. Three red arrows originate from a single point above the index 2 cell and point to the cells containing 92, 57, and 68, demonstrating that elements in an array must be of the same type.

scores :	85	79	92	57	68	80	...		
	0	1	2	3	4	5		98	99

- In some situations, we wish to group elements of different types



A diagram illustrating a structure of different types. A horizontal bar is divided into cells. The first cell contains the text 'employee'. The subsequent cells contain the values 'R. Jones', '123 Elm', '6/12/55', and '\$14.75', followed by a vertical bar '|'. Three red arrows originate from a single point above the first cell after 'employee' and point to the cells containing 'R. Jones', '123 Elm', and '6/12/55', demonstrating that elements in a structure can be of different types.

employee	R. Jones	123 Elm	6/12/55	\$14.75	
----------	----------	---------	---------	---------	--

Structures

7

- RECORDS are used to group related components of different types
- Components of the record are called fields
- In C++

employee	Name	123	6/12/95	PKR 40K
----------	------	-----	---------	---------

- ▣ record called a struct (structure)
- ▣ fields called members

Structures

8

□ C++ **struct**

- ▣ structured data type
- ▣ fixed number of components
- ▣ elements accessed by name, not by index
- ▣ components may be of different types

```
struct part_struct {  
    char descrip [31], part_num [11];  
    float unit_price;  
    int qty; };
```

Declaring struct Variables

9

□ Given

```
struct part_struct {  
    char descrip [31], part_num [11];  
    float unit_price;  
    int qty; };
```

□ Declare :

Use struct name as a type.

```
part_struct new_part, old_part;
```

Declaring struct Variables

10

□ Given

```
struct part {  
    char descrip [31], part_num [11];  
    float unit_price;  
    int qty; };
```

□ Declare :

Use struct name as a type.

```
part new_part, old_part;
```

Accessing Components

11

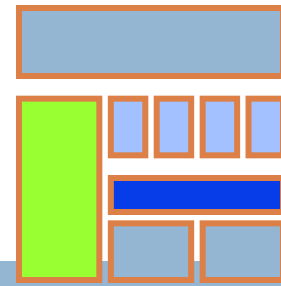
- Use the name of the record
the name of the member
separated by a dot .

```
old_part.qty = 5;
```

```
cout << new_part.descrip;
```

- The dot is called the **member selector**

struct basics



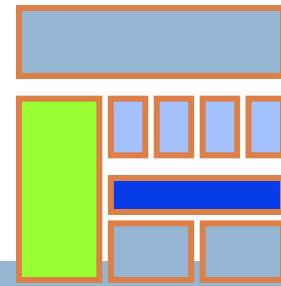
□ Definition of a structure:

```
struct <struct-type>{  
    <type> <identifier_list>;  
    <type> <identifier_list>;  
    ...  
} ;
```

□ Example:

```
struct Date {  
    int day;  
    int month;  
    int year;  
} ;
```

struct examples



□ Example:

```
struct StudentInfo{  
    int Id;  
    int age;  
    char Gender;  
    double CGA;  
};
```

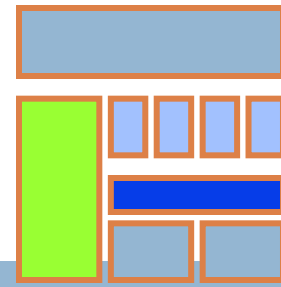


□ Example:

```
struct StudentGrade{  
    char Name[15];  
    char Course[9];  
    int Lab[5];  
    int Homework[3];  
    int Exam[2];  
};
```



struct examples



□ Example:

```
struct BankAccount{  
    char Name[15];  
    int AccountNo[10];  
    double balance;  
    Date Birthday;  
};
```

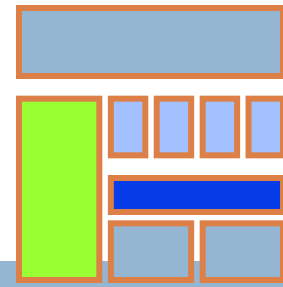


□ Example:

```
struct StudentRecord{  
    char Name[15];  
    int Id;  
    char Dept[5];  
    char Gender;  
};
```



struct basics



- Declaration of a variable of struct type:

```
<struct-type> <identifier_list>;
```

- Example:

```
StudentRecord Student1, Student2;
```

Student1

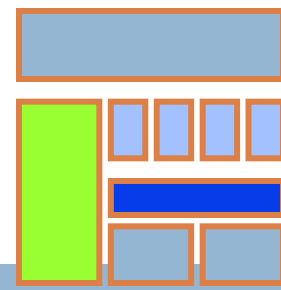
Name	
Id	Gender
Dept	

Student2

Name	
Id	Gender
Dept	

Student1 and **Student2** are variables of **StudentRecord** type.

struct basics

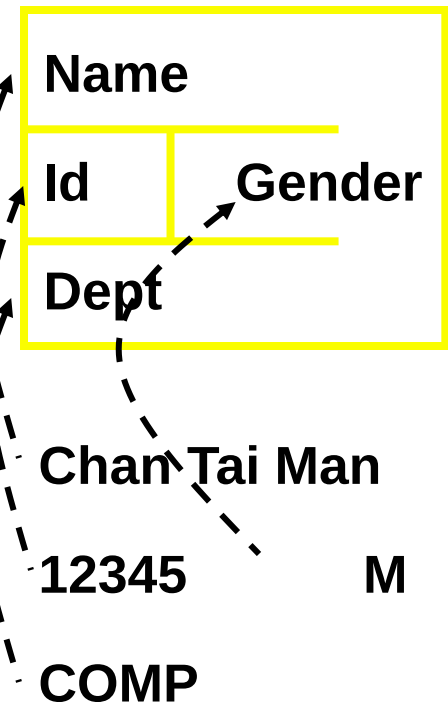


- The members of a **struct** type variable are accessed with the dot (.) operator:

`<struct-variable>.<member_name>;`

- Example:

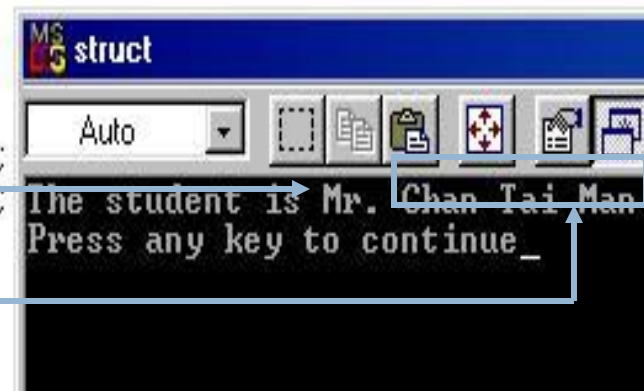
```
strcpy(Student1.Name, "Chan Tai Man");
Student1.Id = 12345;
strcpy(Student1.Dept, "COMP");
Student1.gender = 'M';
cout << "The student is ";
switch (Student1.gender) {
    case 'F': cout << "Ms. "; break;
    case 'M': cout << "Mr. "; break;
}
cout << Student1.Name << endl;
```



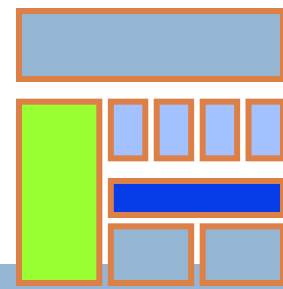
```
#include <string.h>
```

```
struct StudentRecord {  
    char Name[22];  
    int Id;  
    char Dept[22];  
    char gender;  
};
```

```
int main() {  
    StudentRecord Student1;  
  
    strcpy(Student1.Name, "Chan Tai Man");  
    Student1.Id = 12345;  
    strcpy(Student1.Dept, "COMP");  
    Student1.gender = 'M';  
  
    cout << "The student is ";  
    switch (Student1.gender){  
        case 'F': cout << "Ms. "; break;  
        case 'M': cout << "Mr. "; break;  
    }  
    cout << Student1.Name << endl;  
    return 0;  
}
```



struct-to-struct assignment

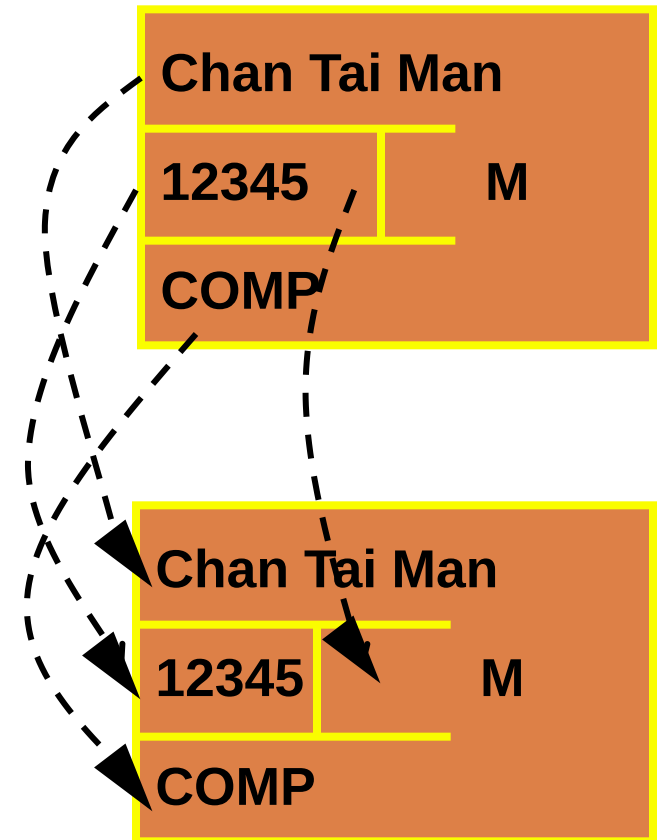


- The values contained in one struct type variable can be assigned to another variable of the same struct type.

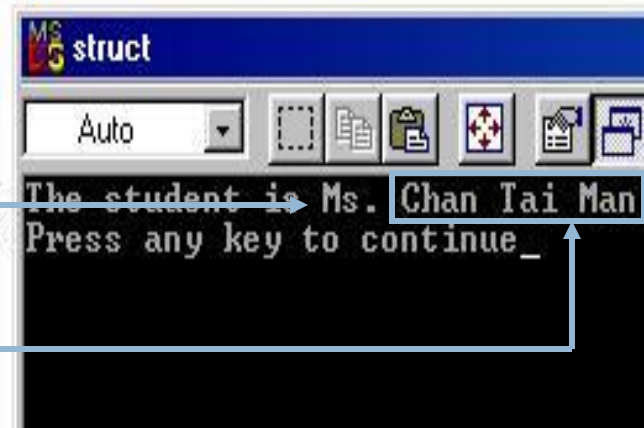
- Example:

```
strcpy(Student1.Name,  
        "Chan Tai Man");  
Student1.Id = 12345;  
strcpy(Student1.Dept, "COMP");  
Student1.gender = 'M';
```

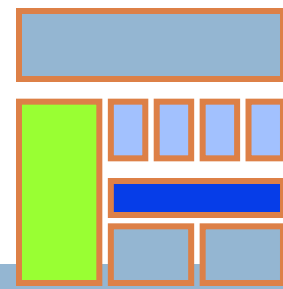
```
Student2 = Student1;
```



```
struct StudentRecord {  
    char Name[22];  
    int Id;  
    char Dept[22];  
    char gender;  
};  
  
int main() {  
    StudentRecord Student1, Student2;  
  
    strcpy(Student1.Name, "Chan Tai Man");  
    Student1.Id = 12345;  
    strcpy(Student1.Dept, "COMP");  
    Student1.gender = 'M';  
  
    Student2 = Student1;  
    Student2.gender = 'F';  
  
    cout << "The student is ";  
    switch (Student2.gender){  
        case 'F': cout << "Ms. "; break;  
        case 'M': cout << "Mr. "; break;  
    }  
    cout << Student2.Name << endl;  
    return 0;  
}
```



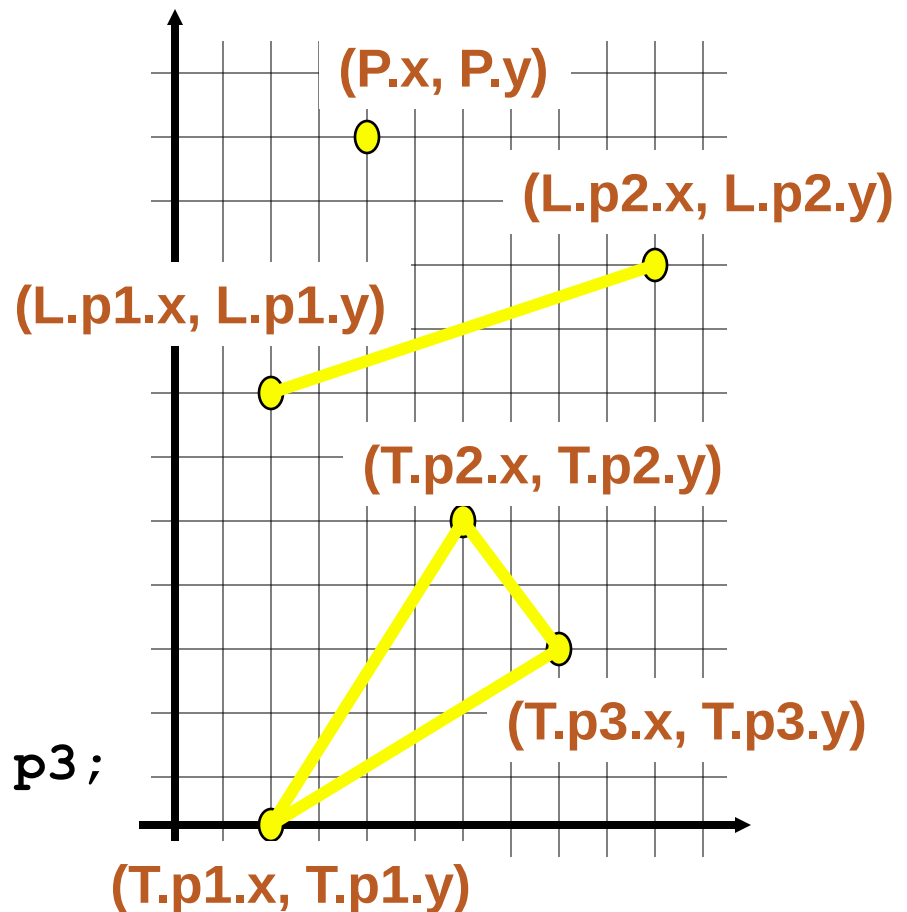
Nested structures



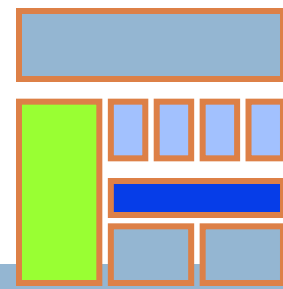
- We can nest structures inside structures.

- Examples:

```
struct point{  
    double x, y;  
};  
point P;  
struct line{  
    point p1, p2;  
};  
line L;  
struct triangle{  
    point p1, p2, p3;  
};  
triangle T;
```

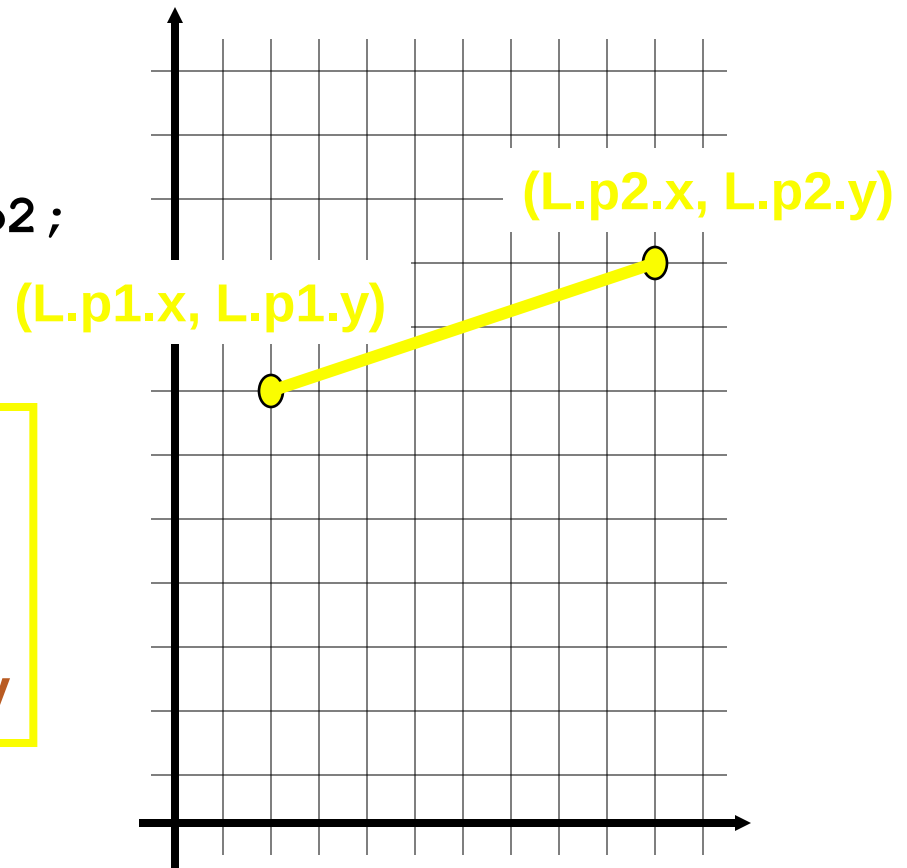
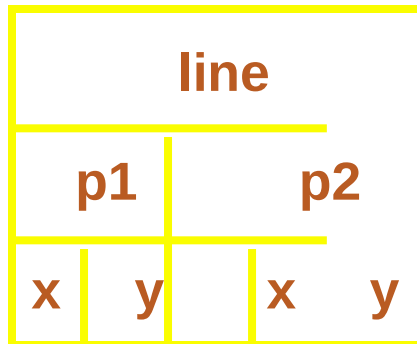


Nested structures

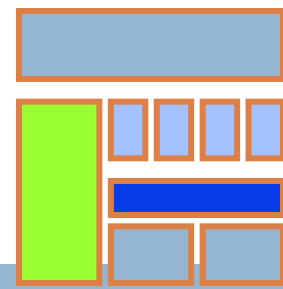


- We can nest structures inside structures.

```
struct line{  
    point p1, p2;  
};  
line L;
```

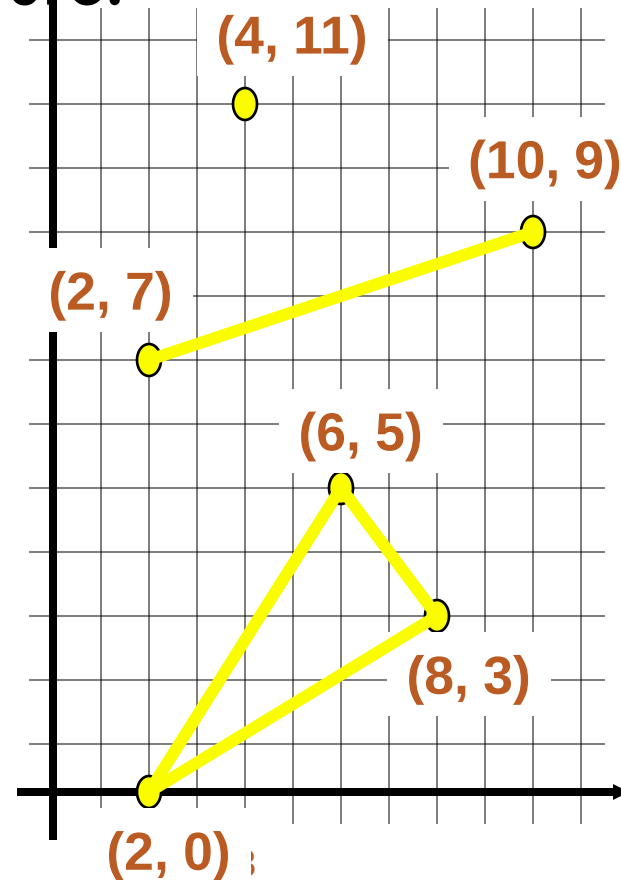


Nested structures

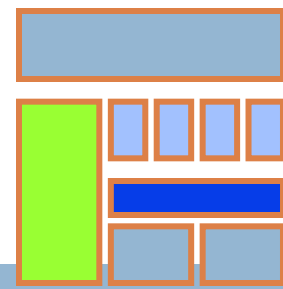


- Assign values to the variables **P**, **L**, and **T** using the picture:

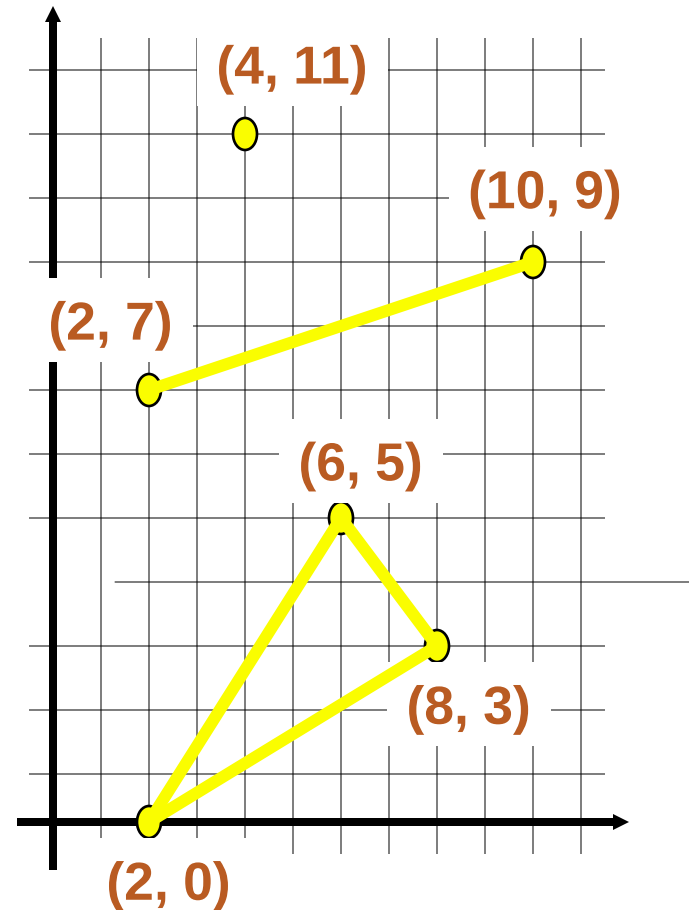
point **P**;
line **L**;
triangle **T**;



Nested structures



```
point P;  
line L;  
triangle T;  
  
P.x = 4;  
P.y = 11;  
L.p1.x = 2;  
L.p1.y = 7;  
L.p2.x = 10;  
L.p2.y = 9;  
T.p1.x = 2;  
T.p1.y = 0;  
T.p2.x = 6;  
T.p2.y = 5;  
T.p3.x = 8;  
T.p3.y = 3;
```



Structure as Functions

```
#include <iostream>
using namespace std;
struct Person
{
    char name[50];
    int age;
    float salary;
};
void displayData(Person); // Function
declaration
int main()
{
    Person p;
    cout << "Enter Full name: ";
    cin.get(p.name, 50);
    cout << "Enter age: ";
    cin >> p.age;
    cout << "Enter salary: ";
    cin >> p.salary;
```

```
// Function call with structure variable as an
argument
displayData(p);
return 0;
}
void displayData(Person p)
{
    cout << "\nDisplaying Information." <<
endl;
    cout << "Name: " << p.name << endl;
    cout << "Age: " << p.age << endl;
    cout << "Salary: " << p.salary;
}
```

Structure as Functions

```
#include <iostream>
```

```
using namespace std;
```

```
struct Person {
```

```
char name[50];
```

```
int age;
```

```
float salary;
```

```
};
```

```
Person getData(Person);
```

```
void displayData(Person);
```

```
int main()
```

```
{
```

```
Person p;
```

```
p = getData(p);
```

```
displayData(p);
```

```
return 0;
```

```
}
```

```
Person getData(Person p) {
```

```
cout << "Enter Full name: ";
```

```
cin.get(p.name, 50);
```

```
cout << "Enter age: ";
```

```
cin >> p.age;
```

```
cout << "Enter salary: ";
```

```
cin >> p.salary;
```

```
return p;
```

```
}
```

```
void displayData(Person p)
```

```
{
```

```
cout << "\nDisplaying Information." <<  
endl;
```

```
cout << "Name: " << p.name << endl;
```

```
cout << "Age: " << p.age << endl;
```

```
cout << "Salary: " << p.salary;
```

```
}
```


Acknowledgements

1. Deitel and Deitel: C++ How to Program, 7th Edition, Prentice Hall Publications
2. Robert Lafore: Object-Oriented Programming in C++, Fourth Edition, December 2001, Sams Publishing .
3. A Structured Programming Approach Using C++ by Behrouz A. Forouzan
4. www.cplusplus.com